Deploy Guide — GCP Cost Observability

This is the **only file you need** to deploy the dashboard, alert pipeline, and daily ETL to any Databricks workspace with Unity Catalog. The whole folder you're reading this in is the installable unit.

---

0. TL;DR — one-command install

```
curl -fsSL https://raw.githubusercontent.com/anujlathiisb/gcp-observability-
databricks/main/install.sh | bash -s -- \
  --workspace-host="https://<your-workspace>.cloud.databricks.com" \
  --profile="my-workspace" \
  --catalog="main" \
  --region="us-central1" \
  --emails="ops@example.com,finance@example.com"
```

`install.sh` clones the repo, installs prerequisites (PyYAML + Databricks CLI if missing), writes `config.yml` from your flags, authenticates with the workspace, and runs `./scripts/deploy.py --target=prod --run-etl`. Two-minute end-to-end.

Run with no flags for an interactive wizard:

```
curl -fsSL https://raw.githubusercontent.com/anujlathiisb/gcp-observability-
databricks/main/install.sh | bash
```

Or run locally from the unpacked folder:

```
./install.sh --workspace-host=https://... --profile=... --catalog=... --region=... --
emails=...
```

Flag reference:

Flag	Env var	Default	Meaning
`--workspace-host`	`WORKSPACE_HOST`	*(prompt)*	Workspace URL
`--profile`	`PROFILE`	*(prompt)*	Databricks CLI profile name
`--catalog`	`CATALOG`	*(prompt)*	UC catalog (must exist)
`--schema`	`SCHEMA`	`observability`	Schema (created by bundle)
`--cloud`	`CLOUD`	`GCP`	`GCP` / `AWS` / `AZURE`
`--region`	`REGION`	`us-central1`	Cloud region for VM list prices
`--threshold`	`THRESHOLD`	`1000`	Daily USD threshold
`--tiers`	`TIERS`	`25,50,75,90,100`	Alert % tiers
`--emails`	`EMAILS`	*(prompt)*	Comma-sep recipient emails
`--tz`	`TZ_NAME`	`Asia/Kolkata`	Timezone for all cron schedules
`--etl-cron`	`ETL_CRON`	`0 0 6 ?`	Quartz cron for daily ETL (6 AM IST)
`--alert-cron`	`ALERT_CRON`	`0 0 * ?`	Quartz cron for alert checker (hourly)
`--gcp-secret-scope`	`GCP_SECRET_SCOPE`	*(empty)*	Secret scope for GCP Billing API key
`--gcp-secret-name`	`GCP_SECRET_NAME`	*(empty)*	Secret name for GCP Billing API key
`--target`	`TARGET`	`prod`	`dev` / `staging` / `prod`
`--repo-url`	`REPO_URL`	embedded	Override repo to clone
`--branch`	`REPO_BRANCH`	`main`	Branch to clone
`--no-run-etl`	`RUN_ETL=false`	(runs ETL)	Skip the initial ETL run
`--yes`	`NO_PROMPT=1`	*(prompt)*	Fail instead of prompting

If you prefer the explicit 3-step flow (edit `config.yml` yourself), keep reading.

---

1. What gets deployed

Running `./scripts/deploy.py --target=prod --run-etl` creates **all of the following** in the target workspace:

Resource	Purpose
UC schema `<catalog>.<schema>` + 6 gold Delta tables	Daily cost + alert history + VM price reference
Job: `[prod] GCP Cost Observability – Daily ETL` (6 AM IST)	Refreshes gold tables + GCP VM reference prices
Job: `[prod] GCP Cost Observability – Hourly Alert Checker` (every hour IST)	Checks daily cost vs threshold, sends email on crossed tier
Lakeview dashboard: `[prod] GCP Cost Observability` (4 pages)	Cost Overview / SKU Deep Dive / GCP VM Costs / Alerts & Config — all month-filterable

Everything is 100% config-driven — no code edits for threshold, recipients, region, timezone, or workspace.

---

2. Prerequisites

On your laptop

- **Databricks CLI v0.230+** — `brew install databricks/tap/databricks`
- **Python 3.9+ with PyYAML** — `pip install pyyaml` (one-time)

In the target workspace

- **Unity Catalog** enabled
- **System tables** enabled (account-admin task):
- `system.billing.usage`, `system.billing.list_prices`
- `system.compute.clusters`, `system.compute.node_types`
- `system.compute.node_timeline` (recommended for accurate VM cost)
- **A serverless SQL warehouse** (any size) — the dashboard uses it
- **An existing UC catalog** you can `CREATE SCHEMA` in (the bundle creates the schema, not the catalog)

Quick smoke-test of system tables in the workspace:

```
SELECT count(*) FROM system.billing.usage WHERE usage_date >= date_sub(current_date(),
7);
SELECT count(*) FROM system.compute.clusters;
```

---

3. Deploy — three steps

Step 1. Authenticate the Databricks CLI to your workspace

```
databricks auth login --host https://<your-workspace-url>
# pick a profile name (e.g. "my-workspace"). You'll reference it in config.yml.
databricks auth profiles | grep my-workspace   # sanity check
```

Step 2. Create and edit `config.yml`

```
cp config.example.yml config.yml
$EDITOR config.yml   # fill in: workspace_host, databricks_profile, catalog, region,
emails
```

Minimum fields to set:

- `workspace_host` — your workspace URL
- `databricks_profile` — the profile name from step 1
- `catalog` — an existing catalog you can write to
- `region` — GCP/AWS/Azure region for VM list prices
- `alert_recipient_emails` — list of emails to notify when a threshold is crossed

Everything else has sensible defaults (IST schedules, 25/50/75/90/100% tiers, `us-central1`, `GCP`).

Step 3. Deploy

```
./scripts/deploy.py --target=prod --run-etl
```

What this does (in order):

1. Writes `local_overrides.yml` with your workspace host + all variables (gitignored)
2. Runs `scripts/prepare.sh` to render the Lakeview dashboard template for your catalog/schema + default month
3. Runs `databricks bundle deploy --target=prod` — uploads notebooks, creates jobs and dashboard
4. Runs `databricks bundle run cost_pipeline` — populates all gold tables immediately

Expected final line:

```
[deploy.py] Done. Target: prod
```

---

4. Verify

```
databricks bundle summary --target=prod --profile=<your-profile>
```

should list the dashboard URL and two jobs.

```
-- Run in the workspace SQL editor
SHOW TABLES IN <catalog>.<schema>;            -- 6 tables
SELECT * FROM <catalog>.<schema>.alert_config; -- threshold + recipients + tiers
SELECT count(*) FROM <catalog>.<schema>.daily_cost_summary;
```

Open the dashboard:

```
databricks bundle open cost_observability_dashboard --target=prod --profile=<your-
profile>
```

Click **Publish** once so other viewers can open it.

---

5. Change configuration later

Option A — edit `config.yml` locally and redeploy (CLI)

```
cd <the-folder-the-installer-cloned>
$EDITOR config.yml
./scripts/deploy.py --target=prod
```

The setup task uses MERGE, so `alert_config` is updated from `config.yml` on every deploy. No SQL needed.

Option B — edit `config.yml` from the Databricks UI and redeploy (no local CLI)

If your laptop doesn't have the CLI anymore, do everything from inside the workspace.

One-time setup — clone the repo as a Databricks Git Folder:

1. Workspace → **Repos** → **Add repo**
2. URL: `https://github.com/anujlathiisb/gcp-observability-databricks` · Branch: `main` → **Clone**
3. Folder appears under `/Workspace/Users/<you>/gcp-observability-databricks`

Edit the config:

1. Open the Git folder in Workspace → click `config.yml`
2. Edit in the workspace editor (same panel the Notebook editor uses) — change threshold, recipients, region, schedule, whatever
3. **Cmd+S** / **Ctrl+S** saves to `/Workspace/Users/<you>/gcp-observability-databricks/config.yml`
4. *(Optional but recommended)* Commit + push via the workspace **Git** icon so the change persists for other environments / teammates

Redeploy from a Web Terminal:

1. Compute → any small cluster → **Web Terminal** tab
2. Run:

```
cd /Workspace/Users/<you>/gcp-observability-databricks
pip install --user pyyaml          # one-time on a fresh cluster
./scripts/deploy.py --target=prod
```

1. Optional: `./scripts/deploy.py --target=prod --run-etl` to kick off an immediate ETL so the new threshold / tiers reflect on the dashboard right away

That's the whole "edit + redeploy from UI" flow. Everything in `config.yml` — threshold, tiers, recipients, region, cron times, timezone, GCP secret pointers — can be changed this way.

Quick runtime tweaks (no redeploy)

For one-off changes between deploys:

```
UPDATE <catalog>.<schema>.alert_config
SET daily_threshold_dollars = 2000,
    recipient_emails = 'new@example.com',
    threshold_percentages = '50,100',
    updated_at = current_timestamp()
WHERE config_id = 1;
```

Note: actual email delivery list is the job's `email_notifications.on_success` which is set from `alert_recipient_emails` at deploy time — change `config.yml` + redeploy to rotate who gets the email. SQL updates affect display + alert_history stamp only.

Optional: live GCP pricing (daily refresh from GCP Cloud Billing Catalog API)

```
# Create a secret scope + secret with your GCP API key
databricks secrets create-scope --scope gcp-billing
databricks secrets put-secret --scope gcp-billing --key api-key --string-value
<YOUR_KEY>

# Point config.yml at them
# gcp_billing_api_key_secret_scope: "gcp-billing"
# gcp_billing_api_key_secret_name:  "api-key"

./scripts/deploy.py --target=prod
```

---

6. File layout (what's in this folder)

```
gcp-cost-observability/
├── DEPLOY.md                      ← this file (canonical guide)
├── README.md                      ← short project overview
├── INSTALL.md                     ← alternative walkthrough
├── config.example.yml             ← copy to config.yml and edit
├── config.yml                     ← YOUR config, not in git
├── databricks.yml                 ← DABs bundle config (don't edit)
├── local_overrides.yml            ← generated by deploy.py (don't edit)
├── scripts/
│   ├── deploy.py                  ← one-command deploy (use this)
│   └── prepare.sh                 ← renders dashboard template (called by deploy.py)
├── resources/
│   ├── cost_pipeline_job.yml      ← Daily ETL job definition
│   ├── alert_pipeline_job.yml     ← Hourly alert job definition
│   └── cost_dashboard.yml         ← Lakeview dashboard resource
└── src/
    ├── 01_setup_schema.py         ← DDL + alert_config upsert from config.yml
    ├── 02_refresh_vm_prices.py    ← Daily VM price refresh (live API or region seed)
    ├── 03_daily_cost_etl.py       ← billing.usage → daily_cost_summary +
sku_daily_details
    ├── 04_vm_cost_estimation.py   ← cluster + VM prices → per-cluster cost rows
    ├── 05_alert_checker.py        ← Threshold eval, writes alert_history
    ├── 06_send_alert_email.py     ← Fires email_notifications.on_success
    └── cost_observability_dashboard.lvdash.json.template  ← rendered by prepare.sh
```

Never edit directly: `databricks.yml`, `local_overrides.yml`, files under `src/` or `resources/`. All user config is in `config.yml`.

---

7. Troubleshooting

Symptom	Cause / Fix
`PyYAML required`	`pip install pyyaml`
`the host in the profile doesn't match the host configured in the bundle`	You ran `databricks bundle deploy` directly without `deploy.py`. Always use `./scripts/deploy.py`.
Dashboard widgets show blank	Re-run `./scripts/deploy.py` so `prepare.sh` re-renders the template for the current month.
Alert emails never arrive	(1) Check `alert_history` has rows for today; (2) Check the `send_alert_email` task ran (not skipped) in the latest `alert_pipeline` run; (3) `alert_recipient_emails` in `config.yml` must be populated; (4) Databricks emails come from `noreply@databricks.com` — check spam
"Today's Spend" stays $0	`system.billing.usage` can lag up to 24h. Also, `current_date()` in Spark is UTC; IST mornings before UTC midnight will show previous day's data
GCP VM Cost = $0 for known compute	Most of your workspace is serverless — Databricks bundles VM into DBU price (marked `vm_cost_source = BUNDLED_IN_DBU`). Classic clusters without a config in `system.compute.clusters` / `node_timeline` are marked `unknown` — that's a Databricks system-tables coverage gap
Cron fires at wrong time	Default timezone is `Asia/Kolkata`. Change `schedule_timezone` in `config.yml` if needed
Month filter doesn't change data	Known Lakeview limitation in this setup: consumer widgets use the deploy-time default month. Re-run `deploy.py` to roll the default forward to the current month; the filter dropdown is cosmetic.

---

8. Uninstall

```
databricks bundle destroy --target=prod --profile=<your-profile>
# Optional: drop the data too
# DROP SCHEMA <catalog>.<schema> CASCADE;
```

Quick reference — daily usage cheat-sheet

```
# Change any config
$EDITOR config.yml && ./scripts/deploy.py --target=prod

# Re-run the ETL now (e.g. after adding a new cluster)
databricks bundle run cost_pipeline --target=prod --profile=<your-profile>

# Trigger the alert checker now (e.g. to test emails)
databricks bundle run alert_pipeline --target=prod --profile=<your-profile>

# Open the dashboard
databricks bundle open cost_observability_dashboard --target=prod --profile=<your-
profile>

# Summary of deployed resources
databricks bundle summary --target=prod --profile=<your-profile>
```